

Introducción a Redes Neuronales Artificiales

Perceptrón, capas, convolución y formulación como problema de optimización

FCFM–UNACH

Licenciatura en Matemáticas Aplicadas

Curso de Aprendizaje Máquina Estadístico

Índice

1. Propósito de esta guía	3
2. Panorama general de una red neuronal	3
3. Tipos de capas en TensorFlow/Keras	4
3.1. Capas densas o fully connected	4
3.2. Capas de convolución	4
3.3. Capas de pooling	5
3.4. Capas recurrentes	5
3.5. Capas de embedding	6
3.6. Capas de normalización	6
3.7. Capas de regularización	6
3.8. Capas de activación	6
3.9. Capas especiales	7
3.10. Capas de salida típicas	7
4. El perceptrón simple	8
5. Interpretación geométrica del perceptrón	8
6. Ejemplo: compuerta AND	9
7. ¿Por qué introducir una función de activación?	9
7.1. Sigmoid	10
7.2. ReLU	10
8. Limitación del perceptrón: XOR	10
9. Del perceptrón al MLP	10
10. Introducción a la convolución bidimensional	11
11. Parámetros fundamentales de Conv2D	11
11.1. filters	11
11.2. kernel_size	11
11.3. strides	11
11.4. padding	11
11.5. activation	11
12. Ejemplo conceptual con una imagen 5×5	12

13. Mini-ejemplo numérico de convolución	12
13.1. Primera ventana	13
13.2. Segunda ventana	13
13.3. Tercera ventana	13
14. De filtros básicos a características complejas	13
15. El entrenamiento como problema de optimización	14
16. Interpretación de los símbolos	14
17. ¿Por qué es un problema sin restricciones explícitas?	14
18. Funciones de pérdida	15
18.1. Regresión	15
18.2. Clasificación binaria	15
18.3. Clasificación multiclase	16
19. Regularización L^2	16
20. Naturaleza del problema de entrenamiento	16
21. Descenso de gradiente	17
22. Gradiente estocástico y minibatches	17
23. Relación con backpropagation	17
24. De una neurona a una CNN: la secuencia conceptual	18
25. Preguntas de autoevaluación	18
26. Ejercicios propuestos	19
27. Actividad integradora	20
28. Ideas esenciales para recordar	20
29. Bibliografía recomendada	21

1. Propósito de esta guía

Las redes neuronales artificiales constituyen una familia de modelos capaces de representar relaciones no lineales mediante composiciones de transformaciones simples. Su estudio puede comenzar desde distintos puntos de vista: como modelos estadísticos, como aproximadores de funciones, como sistemas de capas o como problemas de optimización de alta dimensión.

En esta guía se articulan esas perspectivas siguiendo una secuencia progresiva. Primero se revisan las capas más utilizadas en TensorFlow/Keras. Después se estudia con detalle el perceptrón como unidad elemental de una red. A continuación se introduce la convolución bidimensional como mecanismo fundamental en redes convolucionales. Finalmente, se formula el entrenamiento de una red neuronal como un problema de minimización de una función de pérdida.

Objetivos de aprendizaje

Al finalizar esta guía, el estudiante deberá ser capaz de:

1. Identificar los principales tipos de capas disponibles en TensorFlow/Keras.
2. Explicar qué hace una capa densa y cómo se relaciona con un perceptrón.
3. Interpretar geométricamente la frontera de decisión de un perceptrón.
4. Explicar por qué AND es linealmente separable y XOR no lo es.
5. Comprender el papel de las funciones de activación.
6. Describir el funcionamiento de una capa Conv2D.
7. Realizar manualmente una convolución bidimensional sencilla.
8. Formular el entrenamiento de una RNA como un problema de optimización.
9. Reconocer funciones de pérdida típicas para regresión y clasificación.
10. Relacionar descenso de gradiente, minibatches y optimización estocástica.

2. Panorama general de una red neuronal

Una red neuronal puede verse como una composición de transformaciones. Si la entrada es

$$x \in \mathbb{R}^p,$$

una red con varias capas produce una salida de la forma

$$f_W(x) = f_{W^{(L)}}^{(L)} \circ f_{W^{(L-1)}}^{(L-1)} \circ \cdots \circ f_{W^{(1)}}^{(1)}(x),$$

donde W agrupa todos los pesos y sesgos de la red.

Cada capa transforma una representación en otra. Las primeras capas reciben la información original; las capas intermedias construyen nuevas representaciones; la capa final produce una predicción.

Interpretación

La profundidad de una red no consiste únicamente en “tener muchas neuronas”. Su interés está en la composición sucesiva de transformaciones. Cada capa puede construir características nuevas a partir de las anteriores.

3. Tipos de capas en TensorFlow/Keras

Las capas (**layers**) son los bloques básicos con los que se construye una red neuronal. Cada tipo de capa está diseñado para manipular una estructura particular de datos o introducir una transformación específica.

3.1. Capas densas o fully connected

Una capa densa se crea mediante:

```
1 tf.keras.layers.Dense(  
2     units,  
3     activation="relu"  
4 )
```

Cada neurona de salida se conecta con todas las entradas. Matemáticamente, una capa densa tiene la forma

$$z = Wx + b,$$

seguida, en general, por una activación

$$a = \varphi(z).$$

Se utiliza principalmente en:

- clasificación y regresión con datos tabulares;
- capas finales de CNN;
- capas finales de RNN;
- perceptrones multicapa.

Conexión con aprendizaje máquina

La capa **Dense** es la generalización natural del perceptrón. Por ello conviene comprender primero la geometría de una neurona antes de estudiar arquitecturas profundas.

3.2. Capas de convolución

Ejemplos:

```
1 tf.keras.layers.Conv1D(...)  
2 tf.keras.layers.Conv2D(  
3     filters,  
4     kernel_size,  
5     activation="relu"  
6 )  
7 tf.keras.layers.Conv3D(...)
```

Estas capas aplican filtros locales sobre los datos.
Usos habituales:

- Conv1D: series temporales, audio y secuencias;
- Conv2D: imágenes;
- Conv3D: video y volúmenes tridimensionales.

A diferencia de una capa densa, una convolución explota la estructura local de la entrada y reutiliza el mismo filtro en diferentes posiciones.

3.3. Capas de pooling

Ejemplos:

```
1 tf.keras.layers.MaxPooling2D(  
2     pool_size=2  
3 )  
4  
5 tf.keras.layers.AveragePooling2D(...)  
6  
7 tf.keras.layers.GlobalAveragePooling2D()
```

Estas capas reducen la dimensión espacial de una representación.

El objetivo es resumir información local y disminuir el costo computacional de las capas posteriores.

Punto de atención

Pooling no “aprende” filtros como una convolución. Aplica una operación fija, por ejemplo máximo o promedio, sobre pequeñas regiones.

3.4. Capas recurrentes

Ejemplos:

```
1 tf.keras.layers.SimpleRNN(units)  
2 tf.keras.layers.LSTM(units)  
3 tf.keras.layers.GRU(units)
```

Las capas recurrentes están diseñadas para procesar secuencias. Mantienen un estado interno que resume información de pasos anteriores.

Aplicaciones frecuentes:

- texto secuencial;
- series temporales;
- demanda;
- clima;
- señales.

LSTM y GRU fueron diseñadas para manejar mejor dependencias temporales largas que una RNN simple.

3.5. Capas de embedding

```
1 tf.keras.layers.Embedding(  
2     input_dim,  
3     output_dim  
4 )
```

Una capa de embedding transforma categorías discretas en vectores densos de menor dimensión.

Se utiliza especialmente en:

- procesamiento de lenguaje natural;
- códigos de producto;
- variables categóricas de alta cardinalidad.

3.6. Capas de normalización

Ejemplos:

```
1 tf.keras.layers.BatchNormalization()  
2 tf.keras.layers.LayerNormalization()
```

Estas capas reescalan activaciones y pueden estabilizar el entrenamiento. Son frecuentes en redes profundas, CNN, RNN y Transformers.

3.7. Capas de regularización

Ejemplos:

```
1 tf.keras.layers.Dropout(rate=0.5)  
2  
3 tf.keras.layers.SpatialDropout2D(  
4     rate=0.2  
5 )
```

Durante el entrenamiento, **Dropout** desactiva aleatoriamente una fracción de activaciones. Su objetivo principal es reducir el sobreajuste.

Interpretación

Dropout no elimina neuronas permanentemente. La desactivación ocurre durante el entrenamiento y actúa como una forma de regularización estocástica.

3.8. Capas de activación

Una activación puede especificarse dentro de otra capa o de manera explícita:

```
1 tf.keras.layers.Activation("relu")
```

Funciones frecuentes:

$$\text{ReLU}(z) = \max(0, z),$$

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

$\tanh(z)$,

y **softmax** para clasificación multiclase.

En términos generales:

- ReLU: capas ocultas;
- sigmoid: clasificación binaria;
- softmax: clasificación multiclase;
- tanh: algunas arquitecturas recurrentes.

3.9. Capas especiales

TensorFlow/Keras incluye además capas como:

- **MultiHeadAttention**;
- **Flatten**;
- **Reshape**;
- **Concatenate**;
- **ConvLSTM2D**.

Por ejemplo:

```
1 tf.keras.layers.MultiHeadAttention(  
2     num_heads=8,  
3     key_dim=64  
4 )
```

La atención multi-cabeza es uno de los componentes centrales de los Transformers.

3.10. Capas de salida típicas

Para clasificación binaria:

```
1 Dense(  
2     1,  
3     activation="sigmoid"  
4 )
```

Para clasificación multiclase:

```
1 Dense(  
2     n_clases,  
3     activation="softmax"  
4 )
```

Para regresión:

```
1 Dense(  
2     1,  
3     activation=None  
4 )
```

Actividad para el estudiante

Para cada uno de los siguientes problemas, proponga una capa de salida:

1. predecir la temperatura de mañana;
2. decidir si una imagen contiene o no una lesión;
3. clasificar un dígito entre 10 clases;
4. predecir tres variables reales simultáneamente.

4. El perceptrón simple

El perceptrón de Rosenblatt es uno de los modelos más sencillos de neurona artificial. Para una entrada

$$x = (x_1, \dots, x_n)^T,$$

se calcula primero una combinación lineal

$$z = \sum_{i=1}^n w_i x_i + b.$$

Después se aplica una función de activación f :

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right). \quad (1)$$

Los parámetros son:

- x_i : variables de entrada;
- w_i : pesos;
- b : sesgo;
- f : función de activación.

5. Interpretación geométrica del perceptrón

En dos dimensiones, la frontera de decisión está determinada por

$$w_1 x_1 + w_2 x_2 + b = 0. \quad (2)$$

Esta ecuación describe una recta.

En tres dimensiones describe un plano y, en dimensión general, un hiperplano.

Los pesos controlan principalmente la orientación de la frontera, mientras que el sesgo la desplaza.

Interpretación

El perceptrón simple es un clasificador lineal. Su capacidad geométrica está limitada a separar el espacio mediante un hiperplano.

6. Ejemplo: compuerta AND

La tabla de verdad de AND es:

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

Una frontera ilustrativa es

$$x_1 + x_2 - 1.5 = 0.$$

Equivale a

$$x_2 = -x_1 + 1.5.$$

Los tres puntos

$$(0, 0), \quad (0, 1), \quad (1, 0)$$

quedan de un lado, mientras que

$$(1, 1)$$

queda del otro.

Por tanto, AND es linealmente separable.

Actividad para el estudiante

Verifique el signo de

$$z = x_1 + x_2 - 1.5$$

para cada una de las cuatro entradas. Compruebe que una activación escalón permite reproducir la tabla AND.

7. ¿Por qué introducir una función de activación?

Si varias capas realizaran únicamente transformaciones lineales, su composición seguiría siendo lineal.

Por ejemplo,

$$z_1 = W_1x + b_1,$$

$$z_2 = W_2z_1 + b_2$$

produce

$$z_2 = W_2W_1x + W_2b_1 + b_2,$$

que sigue siendo una transformación afín.

Las funciones de activación rompen esta linealidad y permiten construir modelos mucho más flexibles.

7.1. Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Su salida pertenece a

$$(0, 1).$$

Es natural para representar probabilidades en clasificación binaria.

7.2. ReLU

$$\text{ReLU}(z) = \max(0, z).$$

Es una de las activaciones más utilizadas en capas ocultas de redes profundas.

Conexión con aprendizaje máquina

La no linealidad es lo que permite que una red multicapa construya fronteras de decisión que ya no tienen que ser rectas o hiperplanos.

8. Limitación del perceptrón: XOR

La tabla XOR es:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

No existe una sola recta que coloque simultáneamente los puntos de clase 0 de un lado y los de clase 1 del otro.

Por tanto, XOR no es linealmente separable.

Interpretación

La imposibilidad de resolver XOR con un perceptrón simple motiva la introducción de capas ocultas. Varias neuronas pueden construir regiones intermedias y combinar fronteras lineales para producir una frontera global no lineal.

9. Del perceptrón al MLP

Un perceptrón multicapa (*Multilayer Perceptron*, MLP) combina varias capas densas.

Una arquitectura sencilla puede escribirse como

$$h = \varphi(W_1x + b_1),$$

$$\hat{y} = g(W_2h + b_2).$$

La primera capa transforma la entrada en una representación intermedia h . La segunda utiliza esa representación para producir la salida.

El punto esencial es que la activación no lineal φ permite que la composición represente funciones no lineales.

Actividad para el estudiante

Explique por qué una red con dos capas densas pero sin activación entre ellas sigue siendo equivalente a una sola transformación lineal.

10. Introducción a la convolución bidimensional

Las redes convolucionales se diseñaron para explotar estructura local y regularidades espaciales, especialmente en imágenes.

Una capa `Conv2D` puede construirse mediante:

```
1 tf.keras.layers.Conv2D(  
2     filters=2,  
3     kernel_size=(3,3),  
4     strides=(1,1),  
5     padding="valid",  
6     activation="relu"  
7 )
```

Una imagen puede representarse como una matriz o, si tiene varios canales, como un tensor.

Un filtro o kernel se desplaza sobre pequeñas ventanas de la entrada. En cada posición realiza productos elemento a elemento y después suma los resultados.

11. Parámetros fundamentales de `Conv2D`

11.1. `filters`

Determina cuántos filtros diferentes se aprenden.

Cada filtro produce un mapa de activación.

11.2. `kernel_size`

Determina el tamaño de la ventana local, por ejemplo

$$3 \times 3 \quad \text{o} \quad 5 \times 5.$$

11.3. `strides`

Indica cuántas posiciones se desplaza el filtro en cada paso.

11.4. `padding`

Dos opciones frecuentes son:

- `valid`: no se añaden bordes;
- `same`: se agrega relleno para conservar aproximadamente la dimensión espacial.

11.5. `activation`

Después de la operación lineal se aplica, generalmente, una activación como ReLU.

12. Ejemplo conceptual con una imagen 5×5

Considere la imagen

$$X = \begin{bmatrix} 1 & 2 & 3 & 0 & 1 \\ 0 & 1 & 2 & 3 & 1 \\ 3 & 2 & 1 & 0 & 2 \\ 2 & 0 & 1 & 2 & 3 \\ 1 & 1 & 0 & 2 & 1 \end{bmatrix}$$

y el kernel

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}.$$

Este tipo de kernel responde a cambios de intensidad en una dirección.

Cuando la ventana 3×3 se desplaza sobre la imagen, cada posición produce un nuevo valor.

Si se utilizan dos filtros diferentes, se obtienen dos mapas de activación.

Por ejemplo, una entrada de tamaño

$$28 \times 28 \times 1$$

con

$$2$$

filtros de tamaño 3×3 , `stride=1` y `padding="valid"` produce

$$26 \times 26 \times 2.$$

Interpretación

La dimensión final 2 no corresponde a dos imágenes independientes, sino a dos mapas de características aprendidos a partir de la misma entrada.

13. Mini-ejemplo numérico de convolución

Considere

$$X = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 3 & 1 & 2 & 2 \\ 0 & 1 & 3 & 1 \\ 2 & 0 & 1 & 1 \end{bmatrix}$$

y

$$K = \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}.$$

Usaremos:

$$\text{stride} = 1, \quad \text{padding} = \text{valid}.$$

La salida tendrá tamaño

$$(4 - 2 + 1) \times (4 - 2 + 1) = 3 \times 3.$$

13.1. Primera ventana

Tomamos

$$\begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix}.$$

El producto elemento a elemento con el kernel da

$$(1)(1) + (2)(-1) + (3)(1) + (1)(-1) = 1.$$

13.2. Segunda ventana

$$\begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}$$

produce

$$(2)(1) + (0)(-1) + (1)(1) + (2)(-1) = 1.$$

13.3. Tercera ventana

$$\begin{bmatrix} 0 & 1 \\ 2 & 2 \end{bmatrix}$$

produce

$$(0)(1) + (1)(-1) + (2)(1) + (2)(-1) = -1.$$

Continuando de la misma forma se obtiene

$$Y = \begin{bmatrix} 1 & 1 & -1 \\ -1 & 1 & 1 \\ 1 & -2 & 1 \end{bmatrix}.$$

Si aplicamos ReLU,

$$\text{ReLU}(Y) = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}.$$

Actividad para el estudiante

Verifique manualmente las seis entradas restantes de Y . Después aplique ReLU y explique qué información se elimina.

14. De filtros básicos a características complejas

En una CNN, las primeras capas suelen responder a patrones locales simples, como cambios de intensidad o bordes. Capas posteriores combinan esos patrones y pueden construir representaciones progresivamente más complejas.

La secuencia conceptual es:

$$\text{píxeles} \longrightarrow \text{bordes} \longrightarrow \text{formas} \longrightarrow \text{patrones complejos}.$$

Punto de atención

Los filtros de una CNN entrenada no tienen que coincidir exactamente con detectores manuales de bordes. Durante el entrenamiento, sus coeficientes se ajustan para reducir la función de pérdida.

15. El entrenamiento como problema de optimización

Una red neuronal contiene parámetros que deben aprenderse a partir de datos. Sea

$$\{(x_i, y_i)\}_{i=1}^n$$

un conjunto de entrenamiento.

La red implementa una función

$$f_W : \mathbb{R}^p \longrightarrow \mathcal{Y},$$

donde $W \in \mathbb{R}^d$ agrupa todos los pesos y sesgos.

El entrenamiento puede formularse como

$$\min_{W \in \mathbb{R}^d} \mathcal{L}(W) = \frac{1}{n} \sum_{i=1}^n \ell(f_W(x_i), y_i) \quad (3)$$

donde ℓ es la pérdida individual.

16. Interpretación de los símbolos

Símbolo	Significado matemático	Interpretación en una RNA
$W \in \mathbb{R}^d$	vector de parámetros	pesos y sesgos de todas las capas
$x_i \in \mathbb{R}^p$	vector de entrada	ejemplo de entrenamiento
$y_i \in \mathcal{Y}$	salida deseada	etiqueta, valor o distribución objetivo
f_W	función parametrizada	red neuronal completa
ℓ	pérdida individual	error entre predicción y objetivo
n	tamaño de muestra	número de ejemplos de entrenamiento

17. ¿Por qué es un problema sin restricciones explícitas?

La formulación básica consiste en minimizar

$$\mathcal{L}(W)$$

sobre todo

$$W \in \mathbb{R}^d.$$

No aparecen restricciones explícitas del tipo

$$g(W) = 0$$

o

$$h(W) \leq 0.$$

Sin embargo, la arquitectura impone estructura:

- las dimensiones de cada matriz de pesos dependen de las capas;
- las activaciones determinan la forma funcional del modelo;
- la regularización puede incorporarse al objetivo;
- compartir pesos, como en convoluciones, reduce los grados de libertad efectivos.

Interpretación

La ausencia de restricciones explícitas no significa que el problema carezca de estructura. La arquitectura determina fuertemente qué funciones puede representar la red.

18. Funciones de pérdida

La función de pérdida depende del tipo de problema.

18.1. Regresión

Para una salida real puede utilizarse

$$\ell(f_W(x_i), y_i) = \frac{1}{2} |f_W(x_i) - y_i|^2.$$

El promedio de estos errores produce una versión del error cuadrático medio.

Aplicaciones:

- temperatura;
- precio;
- demanda;
- variables continuas.

18.2. Clasificación binaria

Si

$$y_i \in \{0, 1\}$$

y la salida de la red pertenece a $(0, 1)$, se utiliza frecuentemente la entropía cruzada binaria:

$$\ell(f_W(x_i), y_i) = -[y_i \log f_W(x_i) + (1 - y_i) \log (1 - f_W(x_i))]. \quad (4)$$

Esta pérdida se combina naturalmente con una activación sigmoid en la salida.

18.3. Clasificación multiclase

Con K clases y codificación one-hot,

$$y_i = (y_{i1}, \dots, y_{iK}),$$

puede utilizarse

$$\ell(f_W(x_i), y_i) = - \sum_{k=1}^K y_{ik} \log(f_W(x_i)_k). \quad (5)$$

La capa de salida suele utilizar softmax.

19. Regularización L^2

Una forma común de regularización consiste en penalizar pesos grandes:

$$\mathcal{L}_{\text{reg}}(W) = \frac{1}{n} \sum_{i=1}^n \ell(f_W(x_i), y_i) + \lambda \sum_{\ell} \|W^{(\ell)}\|_2^2. \quad (6)$$

Aquí

$$\lambda > 0$$

controla la intensidad de la penalización.

En Keras puede especificarse mediante:

```
1 kernel_regularizer=  
2     tf.keras.regularizers.l2(lambda_)
```

Interpretación

La regularización modifica el criterio de optimización. Ya no se busca únicamente reducir el error de entrenamiento, sino equilibrarlo con una preferencia por parámetros de menor magnitud.

20. Naturaleza del problema de entrenamiento

El entrenamiento de redes neuronales presenta varias características:

- **Alta dimensión:** d puede ser muy grande.
- **No convexidad:** la composición de capas no lineales produce un paisaje de pérdida complejo.
- **Diferenciabilidad casi en todas partes:** permite utilizar métodos basados en gradiente.
- **Optimización estocástica:** el gradiente suele aproximarse mediante minibatches.

21. Descenso de gradiente

Si pudiéramos calcular el gradiente de toda la muestra, una iteración básica sería

$$W_{t+1} = W_t - \eta \nabla \mathcal{L}(W_t), \quad (7)$$

donde

$$\eta > 0$$

es la tasa de aprendizaje.

La dirección

$$-\nabla \mathcal{L}(W_t)$$

es una dirección local de descenso.

22. Gradiente estocástico y minibatches

Cuando n es grande, se utiliza un subconjunto

$$B_t \subset \{1, \dots, n\}.$$

El gradiente aproximado es

$$g_t = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_W \ell(f_W(x_i), y_i).$$

La actualización se convierte en

$$W_{t+1} = W_t - \eta g_t. \quad (8)$$

Interpretación

El uso de minibatches introduce variabilidad en la trayectoria de optimización. Por eso el descenso de gradiente estocástico puede visualizarse como un descenso “ruidoso” sobre la superficie de pérdida.

23. Relación con backpropagation

Backpropagation no es, por sí mismo, un algoritmo de optimización.

Su función es calcular eficientemente

$$\nabla_W \mathcal{L}(W)$$

mediante la regla de la cadena.

Después, un optimizador como SGD, Adam o RMSprop utiliza esos gradientes para actualizar los parámetros.

Conexión con aprendizaje máquina

Conviene distinguir dos operaciones:

1. **backpropagation**: calcula derivadas;
2. **optimizador**: usa esas derivadas para modificar los parámetros.

24. De una neurona a una CNN: la secuencia conceptual

El contenido de esta guía puede resumirse mediante la siguiente progresión:

$$\boxed{\text{Perceptrón} \longrightarrow \text{MLP} \longrightarrow \text{capas especializadas} \longrightarrow \text{CNN}}$$

El perceptrón introduce:

- pesos;
- sesgo;
- activación;
- frontera de decisión.

El MLP introduce:

- varias capas;
- representaciones ocultas;
- fronteras no lineales.

La CNN introduce:

- estructura local;
- filtros compartidos;
- mapas de activación;
- jerarquías espaciales.

Todas ellas se entrenan mediante la misma idea general:

$$\min_W \mathcal{L}(W).$$

25. Preguntas de autoevaluación

1. ¿Qué diferencia esencial existe entre una capa densa y una capa convolucional?
2. ¿Qué función cumple el sesgo b en un perceptrón?
3. ¿Por qué AND es linealmente separable?
4. ¿Por qué XOR no puede resolverse con un solo perceptrón?
5. ¿Qué aporta una activación no lineal?
6. ¿Qué representan `filters` y `kernel_size` en una `Conv2D`?
7. ¿Qué diferencia existe entre `padding="valid"` y `padding="same"`?
8. ¿Por qué varias capas lineales sin activación siguen representando una transformación lineal?
9. ¿Qué representa W en el problema de entrenamiento?

10. ¿Qué diferencia existe entre función de pérdida y optimizador?
11. ¿Qué papel desempeña backpropagation?
12. ¿Por qué se utilizan minibatches?
13. ¿Qué diferencia existe entre regularización L^2 y Dropout?
14. ¿Qué capa de salida utilizaría para regresión, clasificación binaria y clasificación multiclase?

26. Ejercicios propuestos

Ejercicio 1. Perceptrón AND.

Use

$$w_1 = w_2 = 1, \quad b = -1.5.$$

Calcule z para las cuatro combinaciones binarias y aplique una función escalón.

Ejercicio 2. Fronteras alternativas.

Proponga otros valores de w_1, w_2, b que también separen AND correctamente.

Ejercicio 3. XOR.

Dibuje los cuatro puntos de XOR e intente proponer una recta separadora. Explique geoméricamente por qué no existe.

Ejercicio 4. Composición lineal.

Demuestre que

$$W_2(W_1x + b_1) + b_2$$

puede escribirse como

$$\widetilde{W}x + \widetilde{b}.$$

Ejercicio 5. Convolución manual.

Verifique todas las entradas del mapa

$$Y = \begin{bmatrix} 1 & 1 & -1 \\ -1 & 1 & 1 \\ 1 & -2 & 1 \end{bmatrix}.$$

Ejercicio 6. Tamaño de salida.

Para una imagen 32×32 , calcule el tamaño de salida de una convolución 5×5 , stride 1 y padding valid.

Ejercicio 7. Número de parámetros.

Compare el número de parámetros de una capa densa aplicada a una imagen 28×28 con el de una capa Conv2D con 16 filtros 3×3 .

Ejercicio 8. Pérdidas.

Identifique la pérdida apropiada para:

- a) predecir una variable real;
- b) clasificación binaria;
- c) clasificación entre 10 categorías.

Ejercicio 9. Regularización.

Explique qué efecto espera al aumentar λ en una penalización L^2 .

Ejercicio 10. Minibatches.

Compare conceptualmente batch gradient descent, SGD con un dato y SGD con minibatches.

27. Actividad integradora

Diseñe conceptualmente una red para clasificar imágenes pequeñas en dos clases. Debe especificar:

1. dimensión de la entrada;
2. una o dos capas convolucionales;
3. tamaño de los kernels;
4. activaciones;
5. alguna estrategia de reducción de dimensión;
6. una capa densa final;
7. activación de salida;
8. función de pérdida;
9. optimizador.

Después explique cada decisión utilizando los conceptos desarrollados en esta guía.

Conexión con aprendizaje máquina

La finalidad de esta actividad no es encontrar una única arquitectura correcta, sino aprender a justificar matemáticamente cada bloque de una RNA.

28. Ideas esenciales para recordar

- Una red neuronal es una composición de transformaciones parametrizadas.
- El perceptrón es la unidad elemental de una red densa.
- Un perceptrón simple sólo puede construir fronteras lineales.
- Las activaciones no lineales permiten superar esa limitación.
- Las convoluciones explotan estructura local y reutilizan filtros.
- Pooling reduce dimensión espacial; Dropout regulariza; normalización estabiliza el entrenamiento.
- El entrenamiento consiste en minimizar una función de pérdida respecto de pesos y sesgos.
- Backpropagation calcula gradientes; el optimizador utiliza esos gradientes.
- El uso de minibatches conduce a optimización estocástica.
- La elección de arquitectura, activación, pérdida y optimizador debe ser coherente con el problema.

29. Bibliografía recomendada

Referencias

- [1] F. Rosenblatt, “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain”, *Psychological Review*, 65(6), 1958.
- [2] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
- [3] I. Goodfellow, Y. Bengio y A. Courville, *Deep Learning*, MIT Press, 2016.
- [4] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*, MIT Press, 2012.